

Using LLMs to Animate Interactive Story Characters with Emotions and Personality

Aline Normoyle*
Bryn Mawr College

João Sedoc†
New York University

Funda Durupinar‡
University of Massachusetts Boston

ABSTRACT

Animating performances for story-based games is a difficult and labor-intensive task. Although much research in animation and intelligent agents has focused on the problem of generating animation from textual descriptions, this work explores a novel approach through the use of a Large Language Model (LLM) that animates non-player characters with personality and emotions. The proposed approach thus reduces the typical authoring and setup requirements. As a proof of concept, we demonstrate the approach within a point-and-click narrative game.

Index Terms: Computing Methodologies—Computer Graphics—Animation—Procedural Animation; Human-Centered Computing—Interactive Systems and Tools;

1 INTRODUCTION

Animating performances for story-based games is a difficult and labor-intensive process. These performances, consisting of voice acting, facial expressions, gaze, hands, and body gestures, provide important subtext to conversations with non-player characters (NPCs). Although big-budget game studios can hire actors and skilled animators to create rich performances to enhance and support the story, small studios and non-professional creators cannot. Furthermore, approaches based on pre-recorded performances is not feasible for open-ended procedurally generated content.

This work explores the use of non-verbal cues, such as gaze, arm movements, and facial expressions, for enhancing storytelling based on dialogue with a player. Although there is a considerable body of research on procedural storytelling for games, little work has explored how procedural body language can be used to enhance the generated text with sub-text. Similarly, although much research in animation and intelligent agents has looked at the problem of generating animation based on text, not many have focused on procedural animation. Our work explores a novel approach that reduces the authoring and setup typically needed in narrative scenarios, through the use of a Large Language Model (LLM) that controls the animation of non-player characters with personality and emotions. This approach produces animations when none might be available and allows us to dynamically alter the body language in response to changes in game state, or interactions with the player. Furthermore, this approach is easily integrated with standard 3D characters rigged with facial blend shapes and skeletal body motion by querying an off-the-shelf LLM, GPT-3.5 [20], for the animation parameters that drive the animation.

To demonstrate our approach, we integrate a character into a short narrative point-and-click game. In the game, the player is a microbiologist aboard a shuttle orbiting Mars along with a non-player character (NPC), Lieutenant Eddy Kelly, who is the pilot and

*e-mail: anormoyle@brynmawr.edu

†e-mail: jsedoc@stern.nyu.edu

‡e-mail: funda.durupinarbabur@umb.edu

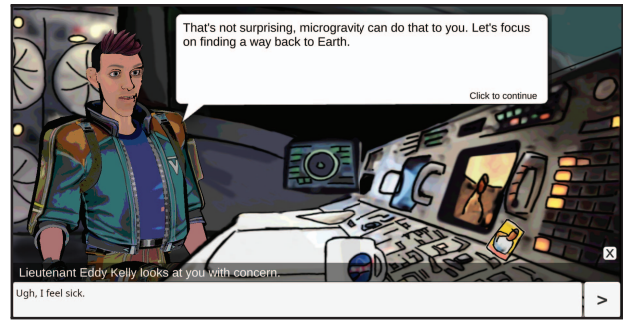


Figure 1: Screenshot of character in our game environment. Above, the character is configured with a cold and distant personality and responds to the player with a concerned expression.

commanding officer. Our proof of concept shows how our animation system can be easily integrated into an interactive narrative using behavior trees, a standard game design approach [18].

2 RELATED WORK

2.1 Generating motion from text

The surge of pre-trained LLMs and large-scale vector-space representations of words have made possible the once unattainable motion synthesis from textual descriptions. This differs from earlier methods that focused on synthesizing gestures to accompany speech. For instance, Bhattacharya *et al.* [3] demonstrate that natural language can be used to create emotionally expressive full-body gestures, extracting the emotional context directly from the input text.

Using a variational autoencoder (VAE) model, Guo *et al.* [10] synthesize diverse motions given a detailed text containing the action type, timing, style, and direction information. Given the scarcity and lack of diversity of datasets for 3D motions with textual descriptions, they have introduced a new dataset, HumanML3D, combining two publicly-available datasets, HumanAct12 [11] and AMASS [15], and having them annotated via crowdsourcing. Jiang *et al.* [12] use the HumanML3D and the earlier KIT [22] datasets, and the LLM T5 [24], to introduce a system that treats motion as a foreign language and allows translating between language descriptions and motion sequence. This system employs a vector quantized VAE architecture and enables the generation of long motion sequences as opposed to the shorter motions of prior studies. Qing *et al.* [23] incorporate trajectories and motion semantics to synthesize long sequences from given long text descriptions. They propose a text-based motion matching framework, which includes a text-driven motion scheduler, a text-driven motion retriever, and neural motion blending components. The LLM works as a motion scheduler in this system.

These works are inherently different from our proof-of-concept method, as they explicitly train machine learning models, use large-scale motion capture and language datasets, and require long training times. We do not assert that our approach achieves the same level of realism as these advanced models. However, our goal is distinct:

we aim to demonstrate that an existing, pre-trained LLM such as GPT-3.5 can be effectively employed to generate procedural motion via clear and instructive textual prompts.

2.2 Interactive story characters in narrative games

Many storylines in games follow a *pearls on a string* approach [25], in which the story unlocks, usually via cutscenes, after periods of open exploration, puzzle solving, and questing. However, this approach typically relies on configuring the animations for characters a priori. Furthermore, this approach is limited in how much the player can influence the story. Research into procedural story generation has investigated how to create open-ended, interactive narratives that reduce authoring and increase replayability. These techniques typically fall into either character-based or plot-based approaches.

With character-based approaches, the narrative is not a pre-written storyline but rather emerges as players and NPCs interact with each other. Examples of this approach in the industry include The Sims [4], or Watch Dogs: Legion [7], which features NPCs with jobs, backstories, and social ties to other NPCs generated by Ubisoft’s “Census System”. Research has additionally explored methods for modeling emotions, social relationships, memories, and personalities [17, 13, 26]. In recent research based on LLMs, Park *et al.* [21] used ChatGPT as a generative backend to simulate a town consisting of 25 agents with unique professions, personalities, and backstories. In a character-based approach, the internal state of the character can be directly used to drive the animation system. However, it is still challenging to configure a 3D animated character that can express the internal state of these agents: we require sufficiently varied motion data sets that are parameterized appropriately.

With plot-based approaches, the plot is procedurally generated. Research has explored techniques based on planning [6], grammars [5], and large-language models [16, 27, 2, 1] for generating stories. From industry, the interactive role-playing engine, AI Dungeon [14], uses an LLM to serve as a dungeon master in a variety of fantasy-themed campaigns. In a plot-driven approach, the appearance and behavior of the characters must be inferred from the text.

3 METHOD

3.1 LLM-Driven Facial Animation

We introduce a novel approach that uses GPT-3.5 prompts to generate facial animation. For this, we use the Facial Action Coding System (FACS), which is a standard for describing facial movement [9]. FACS describes Action Units (AUs) that control the movements of facial muscle groups. AUs are well-defined and can be easily expressed by a combination of blendshapes that most humanoid models use to control their facial rigs.

We first prompt GPT to act as an expert on FACS and analyze the facial expressions of a person for a given input description. The description can include any statement such as “a person who is trying to hide something”, “someone who acts like Mr. Spock”, etc. For our specific use case, our description includes the personality characteristics, the sentence(s) that the character will utter, the duration of the animation, and an optional description of the emotions that will be expressed. The output format is each AU, represented in a row, starting with the AU number, followed by two arrays denoting the times and intensities. The “times” array begins with a 0 and ends with the total duration of the animation sequence in seconds, where each item denotes the keyframes. The “intensities” array denotes the AU’s intensity at each time mentioned in the “times” array. Intensity values range from 0 to 100. We also include the expected output format in the description. E.g.:

AU, Times, Intensities

```
1, [0, 0.1, 0.8, 2.3, 4.5], [0, 100, 40, 60, 0]
12, [0, 0.3, 1.0, 1.5, 4.5], [0, 80, 0, 10, 0]
4, [1.5, 2.0, 3.0, 4.5], [0, 60, 70, 0]
```

... and so forth for each AU.

We map each AU to a group of blendshapes as described in the FACS documentation and based on the model we use. Thus, for instance, AU 1, which controls the inner brow raiser muscles, is characterized by two blendshapes for the left and right brows. Given the key times and intensities, we linearly interpolate the blendshapes for the animation. An extended set of AUs includes head and eye movements. For those, we directly use the corresponding game object transforms, performing spherical linear interpolation from their initial rotations to a target rotation, constrained by the maximum angle for that rotation.

For animating the mouth during speech, GPT-3.5 can return the visemes for each word in a sentence. A viseme is the visual equivalent of a phoneme and describes the mouth movement when speaking a word. Our experiments with GPT-3.5 viseme generation did not return satisfactory results when combined with AU descriptions. Thus, we have decided to use SALSA Lip Sync¹, which is a plugin for Unity² that provides text-to-lip sync. We determine the duration of the utterance provided to GPT-3.5 by considering the “words per minute” parameter of SALSA, multiplied by the number of words in the utterance.

SALSA also provides control for eye gaze. Depending on the personality of the character, we control the tendency to maintain eye contact by setting the target as the main camera and tuning an affinity parameter. Affinity corresponds to the percentage of time the character focuses on the target.

3.2 LLM-Driven Body Animation

Body motion parameters are both more numerous and exhibit greater variability compared to facial control parameters. Therefore, generating motion from language is not as straightforward as tuning AU intensities. So, in this work, instead of having GPT-3.5 control the content of motion, we ask it to infer the style of a given motion via text. For the animation of stylized body movement, we use the PERFORM system [8]. PERFORM applies changes to an input motion based on Laban Movement Analysis (LMA), which is a formal method to categorize qualitative human movement characteristics. LMA’s Effort component specifies a comprehensive parameter set for describing the dynamics of human movement. Effort characterizes the dynamic aspects of motion, describing one’s inner attitude toward four bipolar factors: Space, Weight, Time, and Flow.

Similar to face prompts, we first tell GPT-3.5 to act as an LMA expert and analyze the body movement of a person for a given input description. We then follow with a similar description of the character’s personality and emotions. As output, we ask the Effort values represented in an array in the format *[Space, Weight, Time, Flow]* between -1 and 1, where negative values are indulging (Indirect, Light, Sustained, Free), and positive values are condensing (Direct, Strong, Sudden, Bound). Thus, for instance, when we ask GPT-3.5 to describe the motion of a sociable and friendly person, saying “Hi, nice to meet you!” in a joyful manner, we get the response $[-0.89, -1, 0, -1]$, suggesting Indirect, Light, and Free motion. In contrast, GPT-3.5 returns $[0.93, 0, -1, 1]$ for a cold and distant person uttering the same sentence in a shy manner, indicating Direct, Sustained, and Bound motion. These results are highly consistent with the literature and the perceptual judgments of observers [8].

4 CASE STUDY

As a proof of concept, we have implemented a short interactive game based on the prologue of the episode *One Small Step* from the show *Star Trek: Voyager* [28]. Our game combines our animation system with typical point-and-click adventure game mechanics in which

¹<https://crazyminnowstudio.com/docs/salsa-lip-sync>

²<https://unity.com/>



Figure 2: Utterances and their corresponding facial expressions generated by GPT-3.5. Action Units that have been activated for these utterances are as specified below each image.

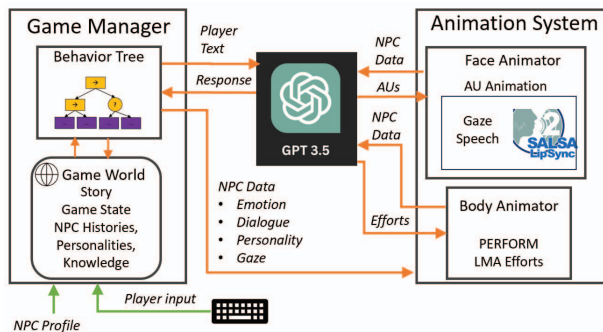


Figure 3: Workflow of the system

the player advances the plot by talking with non-player characters and collecting items. Specifically, the player is a microbiologist on a mission to study Mars when their space shuttle encounters an anomaly that displaces the ship. The player must figure out how to contact Earth with the help of their commanding officer, Lieutenant Eddy Kelly.

Our game implementation consists of two primary modules: the animation system and the game manager (Figure 3). Both the animation system and game manager make queries to GPT-3.5, using an HTTPS connection to OpenAI’s REST API. The animation module controls all NPC movements, including body movement, facial animation, lip-syncing, and eye gaze. The game manager controls the narrative flow and interactions with the player and triggers animations.

4.1 Game Manager

The game manager implements the game scenario using a common game scripting technique based on behavior trees [18]. With his approach, the game manager consists of two primary sub-components: the behavior tree system that executes behaviors according to a loaded script; and a World object that stores all game states that should persist between behaviors. Figure 4 shows an example of a game script. Each line corresponds to a behavior node. Composite behaviors contain all following behaviors until an *End* line is reached. In our system, the root node is always a sequence node, with top-level script nodes as children of the root. The game ends

```
InitCharacter: KELLY
Story: The player is sitting next to Eddy Kelly.
Emotion: KELLY, With a calm and happy expression.
WaitClick: KELLY
  LookAt: KELLY, LookTargetPlayer
  KELLY: Hey Rookie, how are you feeling?
  Dialogue: KELLY
End
```

Figure 4: Example behavior tree script. *InitCharacter* loads personality, history, and knowledge profiles for a character. *Story* appends context for prompts. *LookAt* and *Emotion* set gaze and emotion for the animation system. *WaitClick* pauses until the player clicks on Lieutenant Kelly with mouse. *KELLY* shows dialogue from Lieutenant Kelly. *Dialogue* waits for the player to input text and then shows the response from GPT-3.5.

when this sequence is complete.

Our demo implements special purpose behaviors for explicitly controlling the animation of the character, *Emotion* and *LookAt*. *Emotion* nodes set the current baseline emotion of the character. We also update this emotion based on dialogue with the player. For example, when the player talks to a character, we ask GPT-3.5 for both a dialogue and emotional response. Additionally, using the *InitCharacter* behavior, we load a character profile to use as a starting point. This character profile consists of personality (top five traits), knowledge (domains of expertise), and history (important life events). In combination with the *Story*, which lists the events and facts of the scenario so far, the character profile provides context to the GPT-3.5 that is necessary to ensure sensible and consistent responses to the player. Furthermore, the given personality of a character guides the animation system.

5 CONCLUSION AND FUTURE WORK

This work explores the potential and limitations of using LLMs for animation. LLMs provide an easy-to-integrate solution for authoring animations, especially when the LLM is already being used for open-ended dialogues. Given a context, such as a scenario description, a sentence to be uttered, an emotion, or a personality trait, they can guide an animation without using a complicated model. A background animation controller, whether it tunes blendshapes, rotates bones, or simply selects pre-determined motion parameters can thus directly use the information provided by the LLM. Our work demonstrates a proof-of-concept approach by having GPT reference established models of facial expression and body movement analysis to control the facial blendshapes and the movement style of a 3D model. The only information needed for GPT is to assume expertise in these widely acknowledged face and body motion analysis models. Thus, our game system offers a prospective testbed for evaluating different animation techniques and parameters that implement such models as well as assessing different LLMs. Additionally, in the long term, this work will provide an experimental platform for testing hypotheses about the effects of performance on generated dialogue.

Despite their ease of use, LLMs suffer from prompt sensitivity, lack of fine-tuned control, and configurability. However, the steady advancements in LLMs suggest that these limitations will be overcome soon. Our initial experiments with OpenAI’s Fine-tuning API [19] demonstrated promising results, even when using a limited amount of training data. Our future plans involve investigating fine-tuning and conducting qualitative/quantitative comparisons between the proposed LLM-driven approach and other procedural techniques.

REFERENCES

- [1] A. Alabdulkarim, W. Li, L. J. Martin, and M. O. Riedl. Goal-directed

- story generation: Augmenting generative language models with reinforcement learning. *arXiv preprint arXiv:2112.08593*, 2021.
- [2] P. Ammanabrolu, E. Tien, W. Cheung, Z. Luo, W. Ma, L. J. Martin, and M. O. Riedl. Story realization: Expanding plot events into sentences. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 7375–7382, 2020.
 - [3] U. Bhattacharya, N. Rewkowski, A. Banerjee, P. Guhan, A. Bera, and D. Manocha. Text2gestures: A transformer-based network for generating emotive body gestures for virtual agents. In *2021 IEEE Virtual Reality and 3D User Interfaces (VR)*, pp. 1–10. IEEE, 2021.
 - [4] M. Brown. Emergent storytelling techniques in the sims, 2020. Last accessed on 02/1/2024.
 - [5] K. Compton, B. Kybartas, and M. Mateas. Tracery: an author-focused generative text tool. In *Interactive Storytelling: 8th International Conference on Interactive Digital Storytelling, ICIDS 2015, Copenhagen, Denmark, November 30-December 4, 2015, Proceedings 8*, pp. 154–161. Springer, 2015.
 - [6] I. Dart and M. J. Nelson. Smart terrain causality chains for adventure-game puzzle generation. In *2012 IEEE Conference on Computational Intelligence and Games (CIG)*, pp. 328–334. IEEE, 2012.
 - [7] C. Dragert. Census: The systemic backbone behind play as anyone in ‘watch dogs: Legion’, 2021. Last accessed on 02/1/2024.
 - [8] F. Durupinar, M. Kapadia, S. Deutsch, M. Neff, and N. I. Badler. PERFORM: Perceptual approach for adding OCEAN personality to human motion using Laban movement analysis. *ACM Transactions on Graphics*, 36(1), 2017.
 - [9] P. Ekman, W. V. Friesen, and J. C. Hager. *Facial Action Coding System*. Research Nexus division of Network Information Research Corporation, 2002.
 - [10] C. Guo, S. Zou, X. Zuo, S. Wang, W. Ji, X. Li, and L. Cheng. Generating diverse and natural 3d human motions from text. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5152–5161, 2022.
 - [11] C. Guo, X. Zuo, S. Wang, S. Zou, Q. Sun, A. Deng, M. Gong, and L. Cheng. Action2motion: Conditioned generation of 3d human motions. In *Proceedings of the 28th ACM International Conference on Multimedia*, pp. 2021–2029, 2020.
 - [12] B. Jiang, X. Chen, W. Liu, J. Yu, G. Yu, and T. Chen. Motiongpt: Human motion as a foreign language. *arXiv preprint arXiv:2306.14795*, 2023.
 - [13] M. Kreminski, M. Dickinson, and N. Wardrip-Fruin. Felt: a simple story sifter. In *Interactive Storytelling: 12th International Conference on Interactive Digital Storytelling, ICIDS 2019, Little Cottonwood Canyon, UT, USA, November 19–22, 2019, Proceedings 12*, pp. 267–281. Springer, 2019.
 - [14] Latitude. AI Dungeon, 2023. Accessed July 2023.
 - [15] N. Mahmood, N. Ghorbani, N. F. Troje, G. Pons-Moll, and M. J. Black. Amass: Archive of motion capture as surface shapes. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 5442–5451, 2019.
 - [16] L. Martin, P. Ammanabrolu, X. Wang, W. Hancock, S. Singh, B. Harrison, and M. Riedl. Event representations for automated story generation with deep neural nets. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.
 - [17] M. Mateas and A. Stern. Façade: An experiment in building a fully-realized interactive drama. In *Game developers conference*, vol. 2, pp. 4–8. Citeseer, 2003.
 - [18] I. Millington and J. Funge. *Artificial intelligence for games*. CRC Press, 2018.
 - [19] OpenAI. Fine tuning API, 2023. Last accessed on 01/10/2023.
 - [20] OpenAI. GPT-3.5 API, 2023. Last accessed on 01/10/2023.
 - [21] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
 - [22] M. Plappert, C. Mandery, and T. Asfour. The KIT motion-language dataset. *Big data*, 4(4):236–252, 2016.
 - [23] Z. Qing, Z. Cai, Z. Yang, and L. Yang. Story-to-motion: Synthesizing infinite and controllable character animation from long text. In *SIGGRAPH Asia 2023 Technical Communications*, pp. 1–4, 2023.
 - [24] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.
 - [25] J. Schell. *The art of game design: A deck of lenses*. Schell Games Pittsburgh, 2008.
 - [26] A. Summerville and B. Samuel. Kismet: a small social simulation language. In *Summerville, A., Samuel, B. (2020, September). Kismet: a Small Social Simulation Language. In: 2020 International Conference on Computational Creativity (ICCC), (Casual Creator Workshop). ACC*, 2020.
 - [27] P. Tambwekar, M. Dhuliawala, L. J. Martin, A. Mehta, B. Harrison, and M. O. Riedl. Controllable neural story plot generation via reward shaping. *arXiv preprint arXiv:1809.10736*, 2018.
 - [28] M. Wollaefer, J. Scott, B. Fuller, and M. Taylor. Star Trek: Voyager episode 608: One small step, 1999. Accessed July 2023.